When Does a Tiny Recursive Proposer Help an Exact Solver? An Inductive, Verifier-Gated Study on Constraint Rostering

NIRAJ PANKHANIA
COFORGE
NIRAJ.PANKHANIA@COFORGE.COM

Abstract

Tiny Recursive Models (TRM) [1] and their relatives (HRM [2], PTRM [3]) reach strong puzzle-solving accuracy with sub-10M-parameter networks by recursing a small core over an answer state y and a latent state z. Public demonstrations are confined to puzzles and are transductive (a per-instance identity embedding is learned), which limits transfer to production constraint problems. We study an inductive, identity-free variant on constraint rostering—a fixed-size assignment problem with cheaply checkable hard constraints—under a deliberately constrained CPU-only training budget. We ask a narrow question: when does a tiny recursive proposer, selected by a deterministic verifier and backed by an exact CP-SAT fallback, actually reduce solver load, and when does it not? On a single well-trained configuration we reproduce the TRM ablation (recursion with deep supervision adds $+23$ points over an equal-parameter one-pass control on rostering) and find that verifier-gated best-of-K selection tracks the pass@K oracle (to 91.5%) while the learned Q-head plateaus near 82%. A five-seed study at a cheaper budget shows the proposer is high variance ($48 \pm 8\%$ single-shot valid) but that the verifier-gated hybrid is feasible on 100% of instances in every seed, degrades gracefully under distribution shift (feasibility stays 100% as instances get harder), and avoids the exact solver on the majority of easy instances. The central negative result: the realised end-to-end speedup *falls* with problem size ($1.9\times \to 1.1\times \to 0.9\times$ at $30/48/70$ cells) because proposer accuracy collapses faster than CP-SAT slows at these (solver-easy) scales. An off-the-shelf 0.5B language model, few-shot, produces no feasible rosters. We conclude the verifier-plus-fallback construction is sound and a useful safety wrapper, but that its economic value is conditional on regimes where the exact solver is expensive—which these instance sizes are not.

1 Introduction

Recursive-reasoning models challenge the premise that strong reasoning requires scale. HRM [2] couples two recurrent modules at different timescales; TRM [1] reduces this to a single two-layer network recursed over an answer state y and a latent state z with deep supervision and weight EMA, and reports 45% on ARC-AGI-1 with 7M parameters; PTRM [3] adds test-time stochasticity (per-step Gaussian noise, K parallel rollouts, Q-head selection). These results are reported on abstract puzzles, and the training setups are transductive: each instance carries a learned identity embedding, so the model answers only instances seen in training.

Two gaps follow. First, it is unknown whether the recursion mechanism transfers, inductively, to a constrained industrial problem with a real cost model. Second, the literature recommends deploying such a model as a cheap proposer behind an exact solver, but the economics of that pattern have not been measured. We address both on constraint rostering—assigning workers to shifts subject

[0]Preprint. Under review.

to coverage, rest, and workload rules—chosen because it is a fixed-size assignment problem with a cheap, sound feasibility check and an off-the-shelf exact solver (CP-SAT).

We do not propose a new architecture; our contributions are empirical.

1. An inductive (identity-free) instantiation of a TRM-style model on a non-puzzle constraint task, evaluated on held-out instances (Section 3).
2. A controlled comparison of verifier-gated best-of-K selection against the learned Q-head and majority voting, quantifying the selection gap (Section 5.2).
3. A multi-seed robustness, generalization, and cost study of the proposer-plus-exact-solver hybrid, including a negative result on how realised speedup scales with problem size (Section 5.3).

We report results under two training budgets and are explicit about which claims rest on a single seed versus five seeds.

2 Related Work

Recursive reasoning. TRM [1] and HRM [2] established the tiny-recursion paradigm on Sudoku, Maze and ARC. PTRM [3] is the closest methodological antecedent for our inference procedure: it injects per-step Gaussian noise (σ) and selects the best of K rollouts with the model's Q-head. We adopt PTRM-style noise but replace Q-head selection with a deterministic verifier and add an exact-solver fallback.

Learning-augmented rostering. [4] (DNNTS) train a DNN to guide branch-and-bound for personnel rostering; [5] use a DNN to repair MIP-generated nurse schedules toward hidden acceptability constraints. Closest to our setting, [6] train a transformer on CP-generated feasible nurse schedules and probe learnability, reporting a hierarchy (local per-nurse rules are learned; global cross-nurse rules are not) and proposing the outputs as warm starts for repair. None of these is a recursive reasoning model, and none couples verifier-gated test-time scaling with an exact-solver fallback. Our position is narrow: a tiny recursive proposer, trained inductively, wrapped so that every returned solution is verifier-passed or solver-exact, with the hybrid's economics measured against the solver it is meant to offload.

3 Method

Model. A single core is a 2-layer MLP-mixer [7] (token-mixing plus channel-mixing SwiGLU), recursed over two states: the answer logits y and a latent z. One recursion *block* performs n latent updates $z \leftarrow \mathrm{core}(x_e, y, z)$ followed by one answer update $y \leftarrow \mathrm{core}(x_e, y, z)$, where x_e is the token embedding of the partial assignment. Deep recursion applies T blocks with gradient restricted to the last block (truncated BPTT); deep supervision applies the loss at each of N_{sup} outer steps with a detached carry. We keep a weight EMA and a Q/halt head trained with a binary correctness target. The loss per supervision step is masked cross-entropy over *blank* cells only plus a Q-head BCE term: $\mathcal{L} = \mathrm{CE}_{\mathrm{mask}}(f_O(y), y^\star) + \lambda\,\mathrm{BCE}(\hat{q}, \mathbf{1}[\hat{y} = y^\star])$, $\lambda = 0.5$. Masking is necessary: supervising given cells lets the model copy inputs and collapses blank-cell accuracy.

Inductive conditioning. Unlike TRM/HRM/PTRM we use no per-instance identity embedding; inputs are token and positional embeddings only. The model therefore learns general constraint structure and is tested on instances it never saw.

Verifier-gated inference. Following PTRM we add Gaussian noise $z \leftarrow z + \sigma\,\varepsilon$ at each recursion step and run K parallel rollouts. We select the output using the problem's deterministic feasibility checker rather than the Q-head. We compare four selectors: pass@K (oracle, any rollout correct), verifier@K (first verifier-passing rollout), Q-head best@K, and majority@K.

Hybrid with exact fallback. At deployment the proposer runs K verifier-gated rollouts; if none passes, control falls back to an exact CP-SAT solve [8]. Every returned solution is thus verifier-passed or solver-exact, so the proposer can only change cost, never feasibility.

Data and label consistency. Instances are generated by CP-SAT; a partial assignment is produced by blanking cells of a full solution. Supervised per-cell training requires a unique label, so we filter training and test instances to those with a *unique* completion (verified by CP-SAT solution counting, cutting off at two solutions). On the unfiltered many-completion distribution the model collapses ($\sim$2% valid): per-cell supervision cannot fit a one-to-many map. This is a real limitation, not a tuning artefact.

4 Experimental Setup

Tasks. (i) 4×4 Sudoku, 8 of 16 cells blanked, as a validated CSP control. (ii) Shift rostering on a $W \times D$ grid: each day needs exactly $\lfloor W/3 \rfloor$ day and 1 night shift, no night-then-day transition, and at most $D-1$ shifts per worker; cells are OFF/DAY/NIGHT. The primary grid is 6×5 (30 cells); scaling uses 8×6 (48) and 10×7 (70).

Model and optimisation. Core hidden width 128, 2 layers, n=4 latent updates, T=2 blocks, N_{sup}=4; $\sim$228K parameters. The one-pass control sets n=0, T=1, N_{sup}=1 with the *same* architecture and parameter count. Training: AdamW (lr $= 3 \times 10^{-3}$, weight decay 0.1, $\beta = (0.9, 0.95)$), cosine schedule, gradient clip 1.0, batch 512, EMA decay 0.995; evaluation uses the EMA weights. All runs are CPU-only (Apple silicon, 4 threads). Inference uses σ=0.5 (rostering) / 0.4 (Sudoku) and $K \in \{1, 4, 16, 64\}$; the hybrid uses K=16.

Two budgets. We report two configurations explicitly. *Budget A (single seed, well-trained):* 1400 training instances (Sudoku 2000), $\sim$175 epochs, used for the ablation and the selection study (Sections 5.1–5.2). *Budget B (five seeds, strictly time-budgeted, <10 min/run):* 1000 training instances, 90–165 epochs, used for all robustness, generalization, and economic claims (Section 5.3). The gap between the two (below) is itself evidence of seed/budget sensitivity, and we treat Budget-A numbers as indicative.

5 Results

5.1 Recursion vs. one pass at equal parameters (Budget A, single seed)

Table 1: Recursion with deep supervision vs. an equal-parameter one-pass control ($\sim$228K params). Single seed.

Task	One-pass	Recursive	Δ
Sudoku 4×4 (8 blanks)	81.0%	99.5%	+18.5
Rostering 6×5 (6 blanks)	58.5%	81.5%	+23.0

With architecture and parameter count held fixed, recursion with deep supervision adds 18–23 points (Table 1, Figure 1), consistent with the central TRM claim that the gain is from iterative refinement rather than capacity. As a single-seed result we treat the magnitude as indicative; Budget-B below shows the recursive model's absolute accuracy is seed- and budget-sensitive.

5.2 Selection: verifier vs. Q-head vs. majority (Budget A, single seed)

Table 2: Rostering 6×5 test-time scaling (% feasible). Verifier tracks the pass@K oracle; the Q-head does not.

K	1	4	16	32	64
pass@K (oracle)	81.0	87.5	90.0	91.0	91.5
verifier@K	81.0	87.5	90.0	91.0	91.5
Q-head best@K	81.0	81.5	82.0	79.5	79.5
majority@K	81.0	82.5	82.0	82.0	82.0

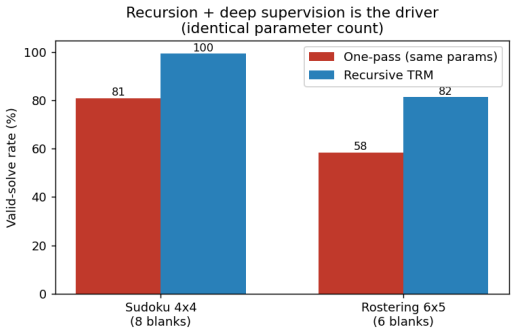


Figure 1: Recursion vs. one pass at equal parameters (Budget A).

A sound verifier recovers the pass@K oracle exactly (both reach 91.5% at $K{=}64$), whereas Q-head and majority selection plateau near 82% (Table 2, Figure 2). For constraint problems, which provide a verifier for free, learned-ranking selection is strictly dominated; we use the verifier throughout.

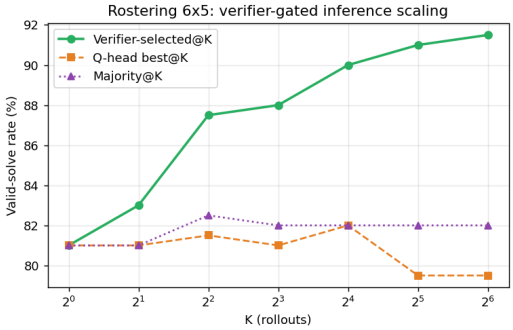


Figure 2: Rostering test-time scaling (Budget A): verifier tracks the oracle, the Q-head plateaus.

5.3 Robustness, generalization, and economics (Budget B, five seeds)

Variance. Across five seeds on 6×5, single-shot valid rate is $48 \pm 8\%$ and verifier@16 is $59 \pm 7\%$ (population std). Hybrid feasibility is 100% in *every* seed (Figure 3a). The spread confirms that single-seed accuracy figures—including Budget A's 81.5%—should not be read as point estimates; absolute accuracy is sensitive to seed and training budget, while the hybrid's feasibility guarantee is not.

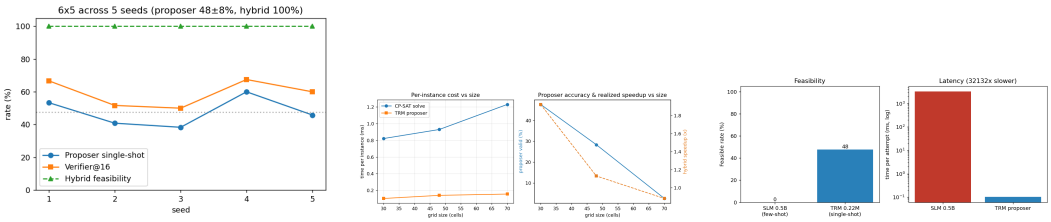
Distribution shift. Training at 6 blanks and testing at 6/8/10, verifier@16 falls $60\% \rightarrow 42\% \rightarrow 34\%$ (3-seed mean), while hybrid feasibility stays 100% throughout: the proposer's hit-rate degrades, but correctness does not, because out-of-distribution instances route to the exact solver.

Economics vs. size. Per attempt the proposer is $\sim$7–8$\times$ cheaper than a CP-SAT solve at all sizes, but the realised end-to-end speedup falls with size (Table 3, Figure 3b): as the grid grows under a fixed cheap budget, proposer accuracy collapses ($48\% \rightarrow 28\% \rightarrow 2\%$) faster than CP-SAT slows, so more instances hit the fallback and at 70 cells the hybrid is slower than calling the solver directly. At these sizes CP-SAT is itself sub-millisecond, so there is little to offload; the pattern pays off only where the exact solver is expensive.

Off-the-shelf SLM baseline. On 6×5, Qwen2.5-0.5B-Instruct [9] prompted few-shot ($n{=}50$, no fine-tuning) produced 0% feasible rosters (52% of outputs even parsed to a complete grid) at 3.3 s/instance: $>$2000$\times$ the proposer's parameters and $\sim$30,000$\times$ its per-attempt latency (Figure 3c). This bounds the zero-shot capability of a general small LM on the task; a fine-tuned SLM would be a stronger, fairer comparator and is left to future work.

Table 3: Cost vs. size (Budget B). Per-instance times in ms; speedup is hybrid vs. solver-only.

Size	Cells	Proposer valid	CP-SAT ms	TRM ms	Realised speedup
6×5	30	48%	0.82	0.10	$1.9\times$
8×6	48	28%	0.93	0.14	$1.1\times$
10×7	70	2%	1.23	0.16	$0.9\times$



(a) Five seeds, 6×5. (b) Cost and speedup vs. size. (c) SLM vs. proposer.

Figure 3: Budget-B study. (a) Hybrid feasibility is 100% on all seeds despite proposer variance. (b) Per-attempt the proposer is cheaper, but realised speedup falls with size and drops below $1\times$ at 70 cells. (c) A 0.5B SLM is 0% feasible and orders of magnitude slower.

6 Discussion and Threats to Validity

The verifier-plus-fallback construction behaves as designed: it is sound by construction and fails safe under shift, and selection by a deterministic verifier strictly dominates the learned Q-head. The useful regime is narrow and identifiable: fixed-size problems with a cheap sound check where the exact solver is the bottleneck. Our instances do not meet the last condition—CP-SAT is sub-millisecond—so the economic case is not demonstrated here and, at 70 cells, is negative. Transfer to regimes where it would matter (larger, solver-expensive instances; soft-objective optimisation rather than feasibility) is untested.

Threats to validity we are aware of: (1) the ablation and selection results are single-seed (Budget A) and the large gap to Budget B shows accuracy is budget/seed-sensitive; (2) absolute accuracy is low under the cheap budget by design; (3) the SLM baseline is zero/few-shot, not fine-tuned; (4) unique-completion filtering restricts the training distribution and is required for supervised per-cell learning; (5) all problems are synthetic and small.

7 Conclusion

An inductive, tiny recursive proposer with verifier-gated selection and an exact- solver fallback reproduces the recursion-is-the-driver result, recovers the pass@K oracle via a verifier where the learned Q-head cannot, and guarantees feasibility under distribution shift. Its economic value, however, is conditional: under a cheap training budget the realised speedup falls with size and turns negative once per-instance solver time is sub-millisecond. The construction is a sound safety wrapper; establishing its economic value requires regimes where the exact solver is expensive, multi-seed training at each size, and a fine-tuned small-LM comparator—the targets of future work.

References

[1] A. Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. *arXiv:2510.04871*, 2025.

[2] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. Abbasi Yadkori. Hierarchical Reasoning Model. *arXiv:2506.21734*, 2025.

[3] Probabilistic Tiny Recursive Model (PTRM). *arXiv:2605.19943*, 2026.

[4] Z. Chen, P. De Causmaecker, and Y. Dou. Neural Networked Assisted Tree Search for the Personnel Rostering Problem. *arXiv:2010.14252*, 2020.

[5] A solution method of nurse scheduling problem using machine learning. *Journal of Advanced Mechanical Design, Systems, and Manufacturing*, 20(5), 2026.

[6] Constraint Learning and Schedule Proposal via Transformers for the Nurse Scheduling Problem. *Proc. PATAT*, 2026.

[7] I. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy. MLP-Mixer: An all-MLP Architecture for Vision. *NeurIPS*, 2021.

[8] L. Perron and F. Didier. OR-Tools CP-SAT Solver. Google, 2024. https://developers.google.com/optimization.

[9] Qwen Team. Qwen2.5 Technical Report. *arXiv:2412.15115*, 2024.